

TaskFlow Frontend Completion Guide

Production frontend implementation prompt for Gemini Pro

Current frontend: Next.js 16, React 19, TypeScript, App Router, Tailwind CSS 4, and Bun.

Security: Never place database credentials, JWT secrets, payment credentials, or real passwords in frontend code or documentation.

Copy-Paste Prompt for Gemini Pro

You are a senior frontend architect and full-stack engineer.

Complete the frontend of my Project Management SaaS application end to end. The frontend must feel like a real production SaaS product, not a demo or template.

Existing project

Frontend location: frontend/

Current stack:

- Next.js 16
- React 19
- TypeScript
- App Router
- Tailwind CSS 4
- Bun package manager

Backend API base URL:

<http://localhost:5000/api/v1>

Production application:

<https://project-management-saas-lemon.vercel.app/>

API documentation:

<https://documenter.getpostman.com/view/49229029/2sBYB4LSM1>

Backend repository:

<https://github.com/DipongkarBarmon/projectmanagementsaasBackend>

Database schema:

<https://drawsql.app/teams/dipongkor-barman/diagrams/projectmanagementsaas>

Rules

1. Inspect the existing frontend before changing anything.
2. Inspect the backend API routes and Postman documentation.
3. Do not invent API endpoints when an existing endpoint is available.
4. Do not use fake static data for completed features.
5. Add loading, empty, error, and success states everywhere.
6. Use strict TypeScript and avoid any.
7. Keep components small, reusable, and readable.
8. Do not duplicate API logic.
9. Do not expose backend secrets.
10. Preserve existing user changes.
11. Run lint and build after implementation.
12. Fix all TypeScript, ESLint, and build errors.
13. Use accessible HTML and keyboard navigation.
14. Make the interface responsive on mobile, tablet, and desktop.
15. Do not create only a marketing landing page. Build the actual application.

Install and use

Install and configure:

- shadcn/ui
- Radix UI
- lucide-react
- @tanstack/react-query
- react-hook-form
- zod
- sonner

- date-fns
- clsx
- tailwind-merge
- next-themes

Use shadcn/ui components and Lucide icons. Use TanStack Query for server state and React Hook Form with Zod for forms.

Visual design

Product name: TaskFlow

Style: minimal, clean, professional, dense, lightly bordered, and responsive.

Font: Inter

Primary: #2563EB

Primary hover: #1D4ED8

Background: #F8FAFC

Card: #FFFFFF

Text: #0F172A

Secondary text: #64748B

Muted text: #94A3B8

Border: #E2E8F0

Success: #16A34A

Warning: #F59E0B

Error: #DC2626

Info: #0EA5E9

Dark theme:

Background #0F172A

Card #1E293B

Border #334155

Text #F8FAFC

Secondary text #94A3B8

Use semantic design tokens instead of scattered raw colors. Use 8px to 12px radius, subtle shadows, strong typography hierarchy, and accessible contrast.

Authentication

Implement registration, login, logout, refresh token handling, email verification, Google authentication, forgot password, reset password, protected routes, authenticated user state, unauthorized page, and session expiration handling.

Use a secure strategy compatible with the backend. Prefer HTTP-only cookies when supported. Never store private secrets in the browser.

Routes

Public:

/login

/register

/verify-email

/forgot-password

/reset-password

/auth/google/callback

Protected:

/dashboard

/projects

/projects/[projectId]

/projects/[projectId]/tasks

/projects/[projectId]/sprints

```
/projects/[projectId]/members  
/teams  
/teams/[teamId]  
/tasks  
/tasks/[taskId]  
/sprints  
/sprints/[sprintId]  
/activity  
/notifications  
/members  
/billing  
/settings  
/settings/profile  
/settings/organization  
/settings/security
```

```
Super admin:  
/admin/billing/plans  
/admin/billing/subscriptions  
/admin/billing/payments  
/admin/billing/payments/[paymentId]
```

Required features

Dashboard:

- Greeting, organization switcher, project/task/member statistics
- Recent projects, task status summary, recent activity
- Upcoming deadlines, notification preview, quick actions

Layout:

- Desktop sidebar about 240 to 260px wide
- Active route highlighting
- Collapsible desktop sidebar
- Mobile navigation drawer
- Organization switcher
- User profile section
- Role-aware navigation
- Top navbar with search, notifications, theme toggle, and user menu

Projects:

- List, create, edit, delete, details, status, progress
- Project manager assignment and member management
- Project tasks, sprints, and activity

Tasks:

- List, table, Kanban board, create, edit, delete, details
- Status, priority, assignee, due date, sprint, labels
- Comments, attachments, activity history

Task statuses:

TODO, IN_PROGRESS, IN_REVIEW, DONE, BLOCKED

Sprints:

- List, create, edit, delete, dates, status, task summary, progress

Sprint statuses:

PLANNED, ACTIVE, COMPLETED, CANCELLED

Teams:

- List, details, create, edit, delete
- Assign leader, add member, remove member

Organizations:

- Details, logo, information editing, members, invitations, switching

Comments and attachments:

- Create, edit, delete comments
- File upload, preview, progress, validation, and deletion

Labels:

- Create, edit, delete, assign, remove, and filter

Notifications:

- List, unread count, mark one read, mark all read, related links

Activity:

- Organization activity, entity activity, actor information, timestamps

Billing:

- Current plan, usage limits, upgrade, downgrade, cancel, resume
- Invoices, payments, payment statuses, bKash redirect handling
- Never expose bKash credentials

API architecture

Create a typed API layer:

```
lib/api/client.ts
lib/api/errors.ts
lib/api/auth.ts
lib/api/organizations.ts
lib/api/invitations.ts
lib/api/projects.ts
lib/api/teams.ts
lib/api/sprints.ts
lib/api/tasks.ts
lib/api/labels.ts
lib/api/comments.ts
lib/api/attachments.ts
lib/api/activities.ts
lib/api/notifications.ts
lib/api/billing.ts
```

Create typed request and response models from the backend. Use TanStack Query for fetching, caching, mutations, invalidation, optimistic updates where useful, retries, and error handling.

Forms

Use React Hook Form, Zod schemas, reusable fields, inline validation, disabled submit states, loading states, success toasts, error toasts, and confirmation dialogs.

Required forms include authentication, organization, invitation, project, team, sprint, task, comment, label, profile, security, and billing forms.

Reusable components

Create reusable components for:

```
Button, Input, Textarea, Select, Combobox, DatePicker, Dialog, AlertDialog, Drawer,
DropdownMenu, Popover, Tooltip, Tabs, Card, Badge, Avatar, Table, DataTable, Pagination,
CommandMenu, Toast, Skeleton, EmptyState, ErrorState, Progress, FileUpload, PageHeader,
StatCard, StatusBadge, PriorityBadge, ProjectCard, TaskCard, TaskTable, KanbanBoard,
```

ActivityItem, and NotificationItem.

Implementation process

Work in phases and validate after each phase.

Phase 1: Foundation

- Install libraries
- Configure shadcn/ui, Tailwind tokens, dark mode, providers
- Add API client, types, global layout, sidebar, and navbar

Phase 2: Authentication

- Login, registration, logout, refresh, protected routes, password recovery, email verification

Phase 3: Dashboard

- Layout, statistics, projects, activity, tasks, notifications

Phase 4: Organizations and teams

- Switching, settings, invitations, teams, members

Phase 5: Projects and tasks

- CRUD, project details, task table, Kanban, task drawer, comments, attachments, labels

Phase 6: Sprints and activity

- Sprint CRUD, board, progress, activity timeline

Phase 7: Billing

- Plans, usage, upgrades, downgrades, cancellation, resume, invoices, payments, bKash flow

Phase 8: Quality

- Loading, empty, error states, mobile, accessibility, lint, TypeScript, production build

After each phase:

1. Explain changed files.
2. Run relevant validation.
3. Fix errors before continuing.
4. Do not silently skip unfinished features.
5. Inspect backend source when an API is ambiguous.

Start by inspecting the current frontend and backend. Implement Phase 1, then continue through all phases until the application is production-ready.

Recommended Frontend Folder Structure

```
frontend/
├─ app/
│   ├── (auth)/
│   │   ├── login/
│   │   ├── register/
│   │   ├── verify-email/
│   │   ├── forgot-password/
│   │   └─ reset-password/
│   ├── (dashboard)/
│   │   ├── layout.tsx
│   │   ├── dashboard/
│   │   ├── projects/
│   │   ├── teams/
│   │   ├── tasks/
│   │   ├── sprints/
│   │   ├── activity/
│   │   ├── notifications/
│   │   ├── members/
│   │   ├── billing/
│   │   └─ settings/
│   ├── admin/
│   │   └─ billing/
│   ├── unauthorized/
│   ├── error.tsx
│   ├── loading.tsx
│   ├── not-found.tsx
│   ├── globals.css
│   ├── layout.tsx
│   └─ page.tsx
├─ components/
│   ├── ui/
│   ├── layout/
│   ├── dashboard/
│   ├── projects/
│   ├── teams/
│   ├── tasks/
│   ├── sprints/
│   ├── comments/
│   ├── attachments/
│   ├── notifications/
│   ├── activity/
│   ├── billing/
│   └─ shared/
├─ hooks/
├─ lib/
│   ├── api/
│   ├── auth/
│   ├── permissions.ts
│   ├── query-client.ts
│   ├── constants.ts
│   ├── formatters.ts
│   └─ utils.ts
├─ providers/
│   ├── auth-provider.tsx
│   ├── query-provider.tsx
│   ├── theme-provider.tsx
│   └─ app-provider.tsx
├─ schemas/
├─ types/
└─ public/
```

```
├─ components.json
├─ .env.local.example
├─ next.config.ts
├─ package.json
├─ tsconfig.json
└─ postcss.config.mjs
```

Environment Variables

```
NEXT_PUBLIC_API_URL=http://localhost:5000/api/v1
NEXT_PUBLIC_APP_URL=http://localhost:3000
```

Only public frontend configuration belongs in `.env.local`. Keep backend secrets on the server.

Validation Checklist

```
bun install
bun run lint
bun run build
```

- Login, registration, logout, and protected routes work.
- Organization switching and role permissions work.
- Project, task, team, sprint, comment, label, and attachment flows work.
- Notifications and billing pages work.
- Dark mode and mobile layouts work.
- There are no TypeScript errors, console errors, unused imports, or fake endpoint calls.

Implementation Order

1. API client and authentication
2. App layout and route protection
3. Organization context and switching
4. Shared UI components
5. Dashboard
6. Projects
7. Tasks and Kanban
8. Teams and members
9. Sprints
10. Comments and attachments
11. Notifications and activity
12. Billing and admin billing
13. Responsive design, accessibility, and production cleanup

Generated for the Project Management SaaS frontend workspace.